

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В.
Г. ШУХОВА» (БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и
автоматизированных систем

Лабораторная работа №3
по дисциплине: Основы ИИ
тема: «Муравьиный алгоритм»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей Александрович

Проверил:
Твердохлеб Виталий Викторович

Белгород 2025 г.

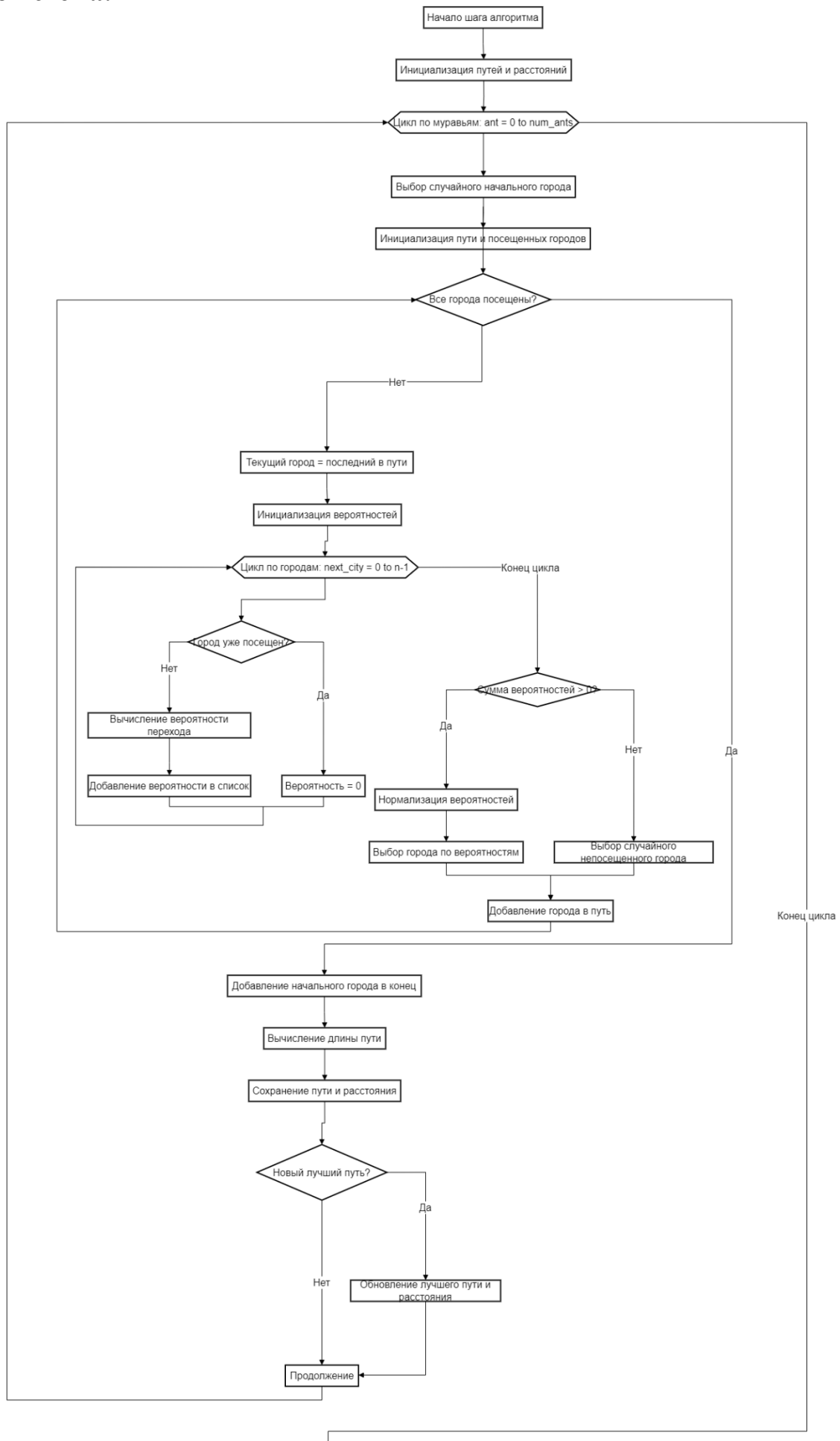
Цель работы: Изучение и исследование параметров муравьиного алгоритма на примере решения задачи коммивояжера.

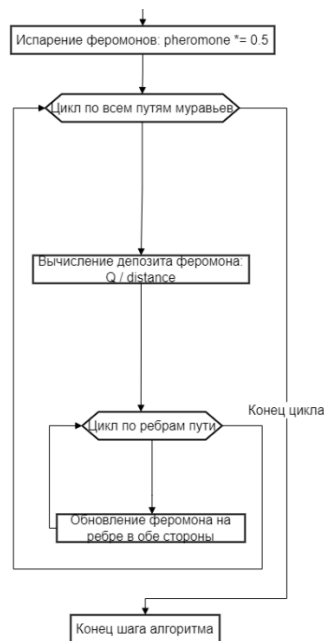
Содержание отчёта:

1. Наименование и цель работы.
2. Блок-схема алгоритма решения задачи N-ферзей с использованием метода обратного восстановления.
3. Исходный код модулей программного обеспечения.
4. Результаты исследования влияния параметров алгоритма на его эффективность, оформленные в виде таблицы.
5. Выводы по работе.

Ход работы:

Блок схема:





Исходный код:

```
def main():
    """Упрощенная версия с анимацией"""
    # Параметры
    num_cities = 30
    num_ants = 15
    num_iterations = 30

    # Генерация данных
    cities = np.random.rand(num_cities, 2) * [800, 600]

    # Вычисление матрицы расстояний
    n = len(cities)
    dist_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
                dist_matrix[i][j] = np.linalg.norm(cities[i] - cities[j])

    # Инициализация
    pheromone = np.ones((n, n))
    history = []
    best_path = None
    best_distance = float('inf')

    # Создание фигуры
    fig, ax = plt.subplots(figsize=(12, 8))

    # Элементы анимации
    best_path_line, = ax.plot([], [], 'green', linewidth=3, label='Лучший
    путь')
    current_path_lines = [ax.plot([], [], 'blue', linewidth=1, alpha=0.3)[0]
    for _ in range(2)]

    # Аннотации городов
    for i, (x, y) in enumerate(cities):
        ax.annotate(str(i), (x, y), xytext=(5, 5), textcoords='offset
    points')

    # Настройка графика - СТАТИЧНЫЙ ЗАГОЛОВОК
    ax.set_xlim(0, 800)
    ax.set_ylim(0, 600)
    ax.set_xlabel('X координата')
    ax.set_ylabel('Y координата')
```

```

ax.set_title('Муравьиный алгоритм', fontsize=14)
ax.grid(True, alpha=0.3)
ax.legend()

def run_algorithm_step(iteration):
    """Выполнение одного шага алгоритма"""
    nonlocal best_path, best_distance, pheromone

    all_paths = []
    all_distances = []

    # Муравьи строят пути
    for ant in range(num_ants):
        current_city = random.randint(0, n - 1)
        path = [current_city]
        visited = {current_city}

        while len(path) < n:
            current = path[-1]
            probabilities = []

            for next_city in range(n):
                if next_city not in visited:
                    prob = (pheromone[current][next_city] ** 1.0) * \
                        ((1.0 / dist_matrix[current][next_city]) **
2.0)
                    probabilities.append(prob)
                else:
                    probabilities.append(0)

            prob_sum = sum(probabilities)
            if prob_sum > 0:
                probabilities = [p / prob_sum for p in probabilities]
                next_city = random.choices(range(n),
weights=probabilities)[0]
            else:
                available = [c for c in range(n) if c not in visited]
                next_city = random.choice(available) if available else
path[0]

            path.append(next_city)
            visited.add(next_city)

        path.append(path[0])
        distance = sum(dist_matrix[path[i]][path[i + 1]] for i in
range(len(path) - 1))
        all_paths.append(path)
        all_distances.append(distance)

        if distance < best_distance:
            best_distance = distance
            best_path = path.copy()
            print(f"Итерация {iteration + 1}: НОВЫЙ ЛУЧШИЙ ПУТЬ! Длина:
{best_distance:.2f}")

        print(f"Итерация {iteration + 1}: Лучшая длина:
{best_distance:.2f}")

    # Обновление феромонов
    pheromone *= 0.5 # Испарение

    for path, distance in zip(all_paths, all_distances):
        deposit = 100 / distance
        for i in range(len(path) - 1):
            pheromone[path[i]][path[i + 1]] += deposit
            pheromone[path[i + 1]][path[i]] += deposit

```

```

# Сохраняем для анимации
return {
    'iteration': iteration,
    'best_path': best_path.copy() if best_path else [],
    'best_distance': best_distance,
    'current_paths': all_paths[:2]
}

def update(frame):
    """Обновление анимации"""
    if frame < num_iterations:
        data = run_algorithm_step(frame)
    else:
        # После завершения всех итераций
        return [best_path_line] + current_path_lines

    # Обновляем лучший путь
    if data['best_path']:
        best_x = [cities[i, 0] for i in data['best_path']]
        best_y = [cities[i, 1] for i in data['best_path']]
        best_path_line.set_data(best_x, best_y)

    # Обновляем текущие пути
    for i, line in enumerate(current_path_lines):
        if i < len(data['current_paths']):
            path = data['current_paths'][i]
            path_x = [cities[city_idx, 0] for city_idx in path]
            path_y = [cities[city_idx, 1] for city_idx in path]
            line.set_data(path_x, path_y)

    # Возвращаем только линии для обновления (без заголовка)
    return [best_path_line] + current_path_lines

print("Запуск муравьиного алгоритма...")
print(f"Параметры: {num_cities} городов, {num_ants} муравьев,
{num_iterations} итераций")
print("-" * 50)

# Запуск анимации
anim = FuncAnimation(fig, update, frames=num_iterations,
                    interval=800, blit=True, repeat=False)

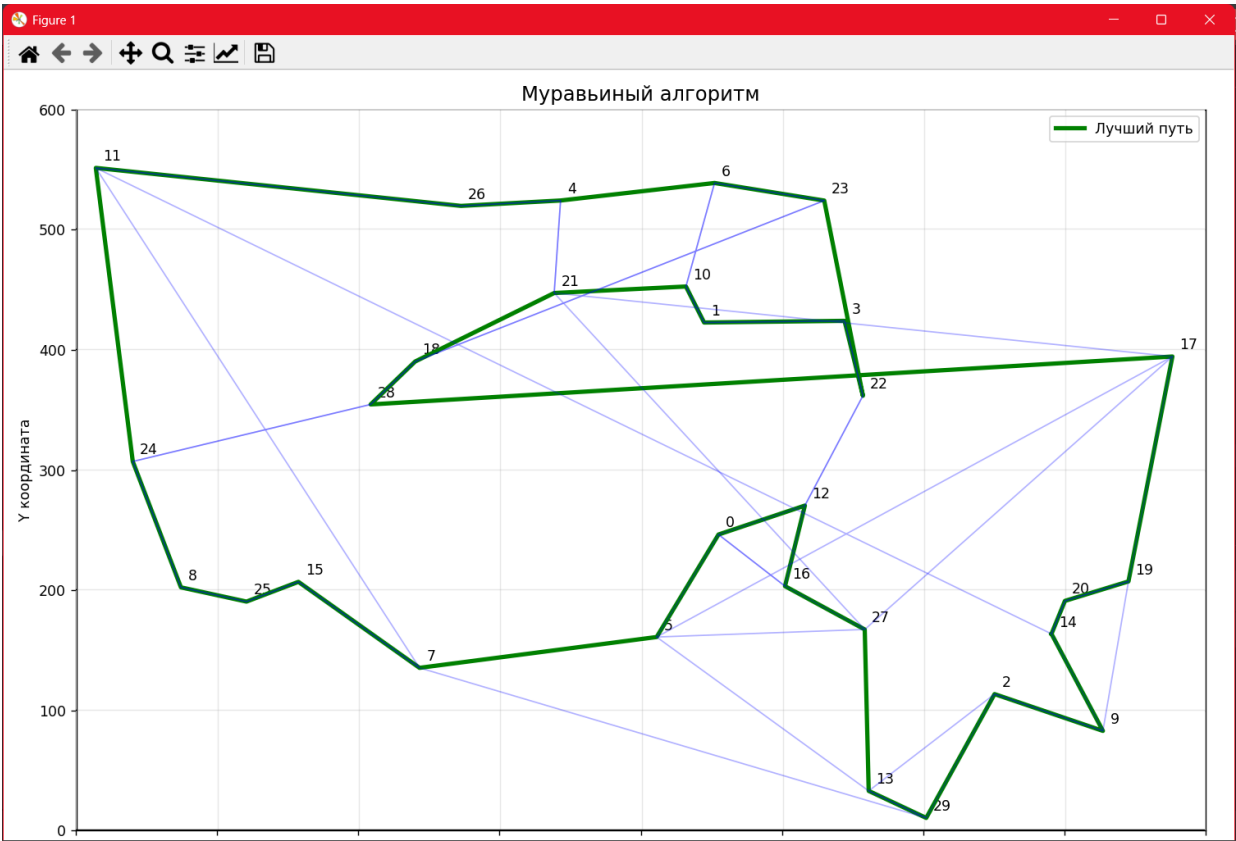
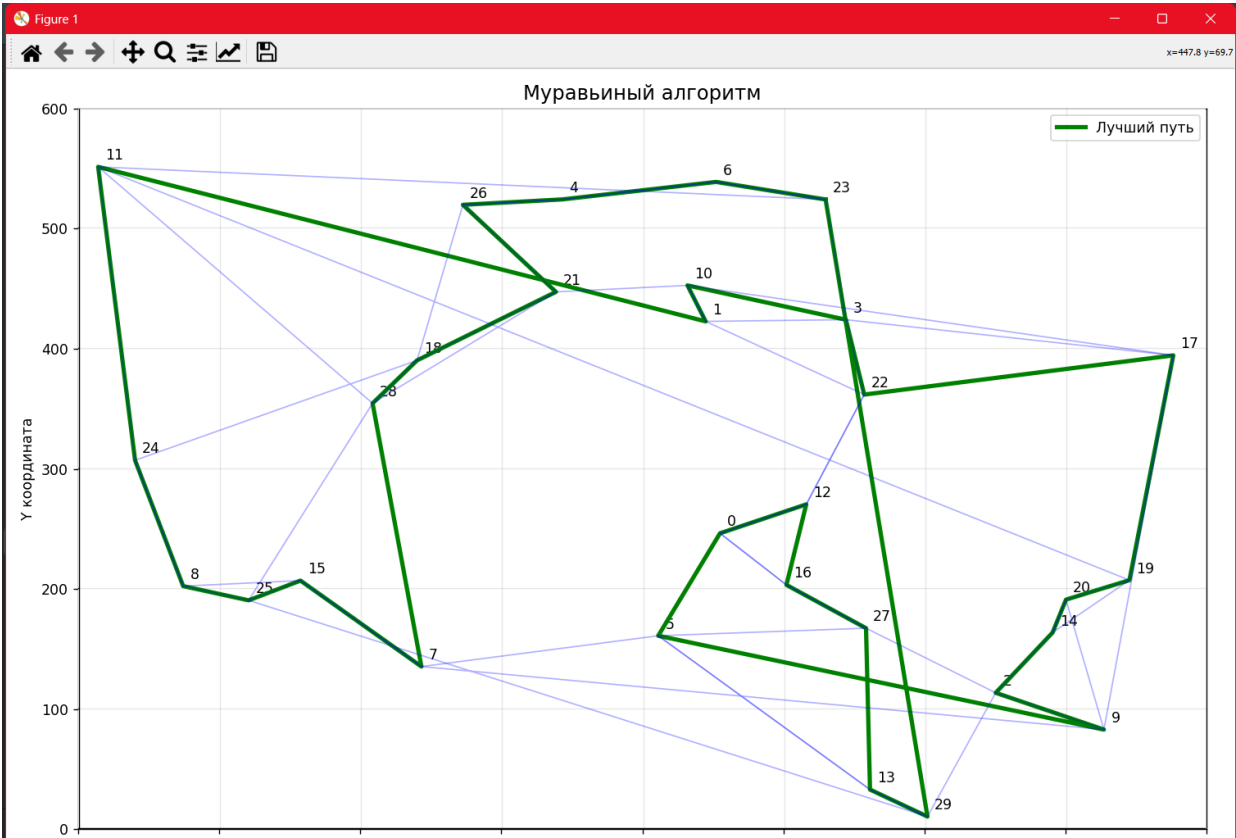
plt.tight_layout()
plt.show()

print("-" * 50)
print(f"Алгоритм завершен!")
print(f"Финальная лучшая длина пути: {best_distance:.2f}")
if best_path:
    print(f"Лучший маршрут: {best_path}")

return anim

```

Пример работы программы:



Эксперименты:

Кол-во городов	a	b	Q	r
20	1	1	3	0.3
20	1	1	3	0.7
20	1	2	5	0.3
20	1	2	5	0.7
20	1	5	5	0.3
20	1	5	5	0.7
20	0.5	5	3	0.3
20	0.5	5	3	0.7
20	0.5	5	5	0.3
20	0.5	5	5	0.7

Вывод: В ходе лабораторной работы выявлено, что алгоритм муравьиной колонии подходит для оптимизации задачи коммивояжёра. Алгоритмы данной группы поддерживает тонкую настройку и подходят для решения широкого спектра задач, так же алгоритмы этой группы легко модифицируются, к примеру при помощи добавления элитных муравьёв или изменения условия окончания работы алгоритма. Для корректной работы данного алгоритма ему требуется правильная настройка, без неё результаты будут далеки от оптимальных.